UNIVERSITÀ DI PISA

DIPARTIMENTO DI INFORMATICA

Corso di laurea in Data Science and Business Informatics

Optimization for Data Science

Solving the Quadratic Min-Cost Flow Problem using the Active Set Method

Carlo Bardazzi, 603930

Anno Accademico 2024-2025

Contents

Contents 1

1 Introduzione 3

2 Active-set Method 3

3 Condizioni di Karush–Kuhn–Tucker (KKT) 4

3.1 Condizioni generali KKT . . . 4

4 Condizione di Ottimalità 5

5 Problemi della matrice Q semidefinita positiva 6

5.1 Problema originale . . . 6

5.2 Sottoproblema . . . 6

5.3 EVD: Eigenvalue Decomposition . . . 6

5.3.1 1. Il Sistema KKT Ristretto . . . 6

6 Calcolo dello Step Size 7

6.1 Aggiornamento della soluzione . . . 7

7 Numero finito di iterazioni 8

7.1 Numero finito di possibili basi (B) . . . 8

7.2 Stretta decrescita della funzione obiettivo . . . 8

7.3 Condizioni di terminazione . . . 8

7.4 Ruolo della stretta convessità . . . 8

8 Pseudocodice 8

9 Risultati 9

9.1 Dataset Grandi . . . 9

9.1.1 Precisione Numerica . . . 9

9.1.2 Prestazioni Computazionali e Scalabilità . . . 9

9.1.3 CPU Time . . . 9

9.1.4 Efficienza Algoritmica . . . 10

9.2 Dataset Casuali (PD e PSD) . . . 10

9.2.1 Precisione Numerica . . . 10

9.2.2 Prestazioni Computazionali e Scalabilità . . . 11

9.2.3 Efficienza Algoritmica . . . 11

A Appendice: Metodo alternativo per trovare la direzione di discesa d 12

A.1 Algoritmo di Pivoting Diagonale per Matrici Semidefinite Positive 12
A.2 Determinazione della Direzione Infinita nel Caso Singolare 12
A.2.1 Determinazione della Matrice D 13
A.2.2 Risoluzione del Sistema nel Caso Singolare 13

1. Introduzione

Consideriamo il seguente problema di ottimizzazione quadratica:

$$(P) \quad \min \left\{ \tfrac{1}{2} x^T Q x + q^T x \ : \ Ex = b, \ 0 \leq x \leq u \right\}.$$

Dove:

- $x \in \mathbb{R}^m$ è un vettore delle variabili decisionali;
- $Q \in \mathbb{R}^{m \times m}$ è una matrice diagonale semidefinita positiva;
- $q \in \mathbb{R}^m$ è un vettore dei coefficienti lineari della funzione obiettivo;
- $E \in \mathbb{R}^{n \times m}$ è la matrice di incidenza di un grafo connesso e orientato con n nodi e m archi. In particolare:

$$E_{ij} = \begin{cases} 1 & \text{se l'arco } j \text{ esce dal nodo } i, \\ -1 & \text{se entra nel nodo } i, \\ 0 & \text{altrimenti.} \end{cases}$$

- $b \in \mathbb{R}^n$ è il vettore dei bilanci dei nodi;
- $u \in \mathbb{R}^m$ è il vettore dei valori massimi di capacità per ciascun arco, con valori positivi.

2. Active-set Method

Consideriamo il nostro problema sotto questa forma:

$$\min_x \ \tfrac{1}{2} x^T Q x + q^T x,$$

soggetto ai vincoli:

$$E_i x = b_i, \ i \in E, \qquad 0 \leq x_i \leq u_i, \ i \in I.$$

Con i vincoli di capacità $0 \leq x \leq u$, un vincolo di bound è *attivo* in x se $x_i = 0$ (limite inferiore) oppure $x_i = u_i$ (limite superiore). In tal caso, la variabile è bloccata al proprio estremo e le direzioni ammissibili devono rispettare:

$$x_i = 0 \Rightarrow d_i \geq 0, \qquad x_i = u_i \Rightarrow d_i \leq u_i.$$

Un vincolo è *inattivo* se $0 < x_i < u_i$ e, di conseguenza, non restringe localmente le direzioni di movimento.

Nel metodo *active-set* si trattano come uguaglianze soltanto i vincoli attivi (il cosiddetto *working set*), mentre i vincoli inattivi vengono ignorati nel calcolo della direzione di ricerca. Nei problemi convessi, le condizioni KKT sono necessarie e sufficienti: i moltiplicatori dei vincoli inattivi sono nulli, quindi focalizzarsi sui vincoli attivi non altera la soluzione ottima.

Per incorporare questi vincoli nel problema, modifichiamo la matrice E e il vettore b includendo tali vincoli. In questo modo otteniamo una formulazione aggiornata del problema con soli vincoli di uguaglianza:

$$\min_x \ \tfrac{1}{2} x^T Q x + q^T x,$$

soggetto a:

$$E_i x = b_i, \ i \in W,$$

dove l'insieme di lavoro è $W = E \cup A(x)$.

Un requisito importante che imponiamo al working set W_k, nella formulazione del metodo *primal active-set*, è che i gradienti a_i dei vincoli inclusi in W_k siano linearmente indipendenti. Questo vincolo è fondamentale per garantire che il sottoproblema quadratico risolto a ogni iterazione sia ben posto e ammetta una soluzione univoca. In particolare, anche se nel punto corrente x_k l'insieme completo dei vincoli attivi (cioè quelli per cui $a_i^T x_k = b_i$) può contenere gradienti linearmente dipendenti, il sottoinsieme selezionato come working set deve sempre essere costituito da vincoli con gradienti linearmente indipendenti. Tale scelta consente di determinare correttamente una direzione di discesa valida e di garantire la stabilità numerica dell'algoritmo.

Indichiamo con $B \subseteq N = \{1, \ldots, n\}$ l'insieme degli indici dei vincoli attivi.

Definiamo quindi:

$$B = (L, U), \qquad L \cap U = \varnothing, \quad L \cup U \subset N, \quad F = N \setminus (L \cup U),$$

dove:

- L rappresenta le variabili fissate al limite inferiore,
- U quelle fissate al limite superiore,
- F contiene le variabili libere.

In questo modo la variabile x può essere scritta come:

$$x = [x_L, x_F, x_U],$$

e risulta quindi che solo x_F è effettivamente "libero".

Senza perdita di generalità, possiamo effettuare una traslazione fissando $\bar{x} = 0$, cioè ponendo $x \leftarrow x + \bar{x}$ e avendo $x_L = 0$. Otteniamo quindi:

$$x = [0, x_F, x_U].$$

Il problema ridotto diventa allora:

$$(P_B) \quad \min \left(\tfrac{1}{2} x_F^T Q_{FF} x_F + (q_F + x_U^T Q_{UF}) x_F\right) + \left[\tfrac{1}{2} x_U^T Q_{UU} x_U + q_U^T x_U\right],$$

soggetto ai vincoli:

$$E_F x_F = b - E_U x_U, \qquad 0 \leq x_F \leq u_F.$$

Questo nuovo problema è non vincolato e si svolge in uno spazio molto più piccolo.

3. Condizioni di Karush–Kuhn–Tucker (KKT)

Sia x^* una soluzione ottimale del seguente problema di ottimizzazione:

$$\begin{cases} \min f(x), \\ g_i(x) \leq 0, \quad i \in I, \\ h_j(x) = 0, \quad j \in J. \end{cases}$$

Se $T_X(x^*) = D_X(x^*)$, allora esistono $\lambda \in \mathbb{R}_+^{|A(x^*)|}$ e $\mu \in \mathbb{R}^{|J|}$ tali che:

$$\nabla f(x^*) + \sum_{i \in A(x^*)} \lambda_i \nabla g_i(x^*) + \sum_{j \in J} \mu_j \nabla h_j(x^*) = 0.$$

Dove:

- x^* è il punto ottimo;
- $f(x)$ è la funzione obiettivo da minimizzare;
- $g_i(x) \leq 0$ sono i vincoli di disuguaglianza, $i \in I$;
- $h_j(x) = 0$ sono i vincoli di uguaglianza, $j \in J$;
- $T_X(x^*)$ e $D_X(x^*)$ sono rispettivamente i derivati tangenti e direzionali in x^*;
- λ e μ sono i moltiplicatori di Lagrange associati ai vincoli attivi e ai vincoli di uguaglianza.

Questa condizione caratterizza l'ottimalità di x^*, implicando l'esistenza di moltiplicatori di Lagrange λ e μ che soddisfano le condizioni di Karush–Kuhn–Tucker (KKT).

Se il problema è strettamente convesso , l'ottimo x^* è unico e globale. Se invece il problema non è convesso, le condizioni KKT descrivono ottimi locali.

3.1. Condizioni generali KKT

Esistono $\lambda \in \mathbb{R}_+^{|A(x^*)|}$ e $\mu \in \mathbb{R}^{|J|}$ tali che:

$$\begin{aligned} & g_i(x) \leq 0, && \forall i \in I, \\ & h_j(x) = 0, && \forall j \in J, \\ & \nabla f(x) + \sum_{i \in I} \lambda_i \nabla g_i(x) + \sum_{j \in J} \mu_j \nabla h_j(x) = 0, && \\ & \lambda_i g_i(x) = 0, && \forall i \in I. \end{aligned}$$

Per il nostro problema specifico, le condizioni di Karush–Kuhn–Tucker (KKT) si possono scrivere come segue.

Condizione di stazionarietà

$$Q_{FF}x_F + \left(q_F + x_U^T Q_{UF}\right) + E_F^T \mu_F = 0 \tag{1}$$

dove:

- E_W è la matrice delle righe di E corrispondenti ai vincoli attivi,
- μ è il vettore dei moltiplicatori di Lagrange.

Condizione di primal feasibility

$$E_F x_F = b - E_U x_U \tag{2}$$

Il sistema può essere riscritto nella forma:

$$\begin{bmatrix} Q_{FF} & E_F^T \\ E_F & 0 \end{bmatrix} \begin{bmatrix} x_F \\ \mu_F \end{bmatrix} = \begin{bmatrix} -\tilde{q} \\ \tilde{b} \end{bmatrix} \tag{3}$$

dove:

$$\tilde{b} = b - E_U x_U, \qquad \tilde{q} = -q_F - x_U^T Q_{UF}.$$

e il sistema (3) rappresenta la forma compatta delle condizioni KKT per il sottoproblema quadratico.

4. Condizione di Ottimalità

Nel contesto del problema (P_B), sia x_F^* la soluzione del sottoproblema e

$$x^* = [0,\, x_F^*,\, x_U]$$

la soluzione completa.

Per caratterizzare l'ottimalità, consideriamo i moltiplicatori di Lagrange associati ai vincoli attivi, indicati con $\bar{\mu}$.

Esistono due tipi di moltiplicatori:

- μ_i per i vincoli di tipo inferiore $x_i \geq 0$, con $i \in L$;
- μ_i per i vincoli di tipo superiore $x_i \leq x_i^*$, con $i \in U$.

L'interpretazione di questi moltiplicatori deriva direttamente dai valori del gradiente della funzione obiettivo calcolata in x^*, indicati con:

$$g_i = [\nabla f(x^*)]_i.$$

Le condizioni di Karush–Kuhn–Tucker (KKT-G) per il problema impongono che:

- Per $i \in L$, abbiamo:
$$g_i - \mu_i = 0 \;\Rightarrow\; \mu_i = g_i.$$
Poiché i moltiplicatori devono essere non negativi ($\mu_i \geq 0$), segue che $g_i \geq 0$ per i vincoli inferiori.
- Per $i \in U$, invece:
$$g_i + \mu_i = 0 \;\Rightarrow\; \mu_i = -g_i.$$
Poiché anche in questo caso i moltiplicatori devono essere non negativi, otteniamo che $g_i \leq 0$ per i vincoli superiori.

Queste condizioni garantiscono che non sia possibile migliorare la soluzione senza violare i vincoli. Infatti:

- Se $g_i < 0$ per qualche $i \in L$, è possibile effettuare un piccolo passo lungo la direzione del gradiente negativo per migliorare la funzione obiettivo senza uscire dalla regione ammissibile. Ciò significa che x^* non può essere ottimale.
- Simmetricamente, se $g_i > 0$ per qualche $i \in U$, si può ridurre x_i senza uscire dalla regione ammissibile e ottenere un valore migliore della funzione obiettivo. Anche in questo caso, x^* non è ottimale.

La condizione di ottimalità richiede che nessuno dei due casi sopra si verifichi, ovvero:

$$g_i \geq 0 \quad \text{per } i \in L, \qquad g_i \leq 0 \quad \text{per } i \in U.$$

Se questa condizione è soddisfatta, significa che nessun passo migliorativo è possibile senza violare i vincoli, e quindi x^* è una soluzione ottimale.

5. Problemi della matrice Q semidefinita positiva

5.1. Problema originale

Una complicazione che possiamo riscontrare avendo la matrice Q semidefinita positiva è che il problema originale possa essere non limitato inferiormente.

Se α è finito, non ci sono problemi di non limitatezza. Se invece $\alpha = \infty$, potrebbero insorgere difficoltà, in particolare se $Ed \leq 0$, poiché in tal caso la funzione obiettivo potrebbe continuare a decrescere senza mai raggiungere un valore minimo, indicando che il problema originale non è limitato inferiormente.

Tuttavia, la presenza dei vincoli di box, che impongono limiti inferiori e superiori su ciascuna variabile, impedisce di percorrere indefinitamente tale direzione. Di conseguenza, l'ampiezza massima del passo α lungo d risulterà comunque finita, in quanto determinata dall'intersezione con i vincoli di box che definiscono la regione ammissibile.

5.2. Sottoproblema

Nel metodo *Active-Set*, la risoluzione del sottoproblema quadratico prevede la soluzione del seguente sistema lineare:

$$\begin{bmatrix} Q_{FF} & E_F^T \\ E_F & 0 \end{bmatrix} \begin{bmatrix} x_F \\ \mu_F \end{bmatrix} = \begin{bmatrix} -\tilde{q} \\ \tilde{b} \end{bmatrix} \tag{4}$$

Questo sistema ammette una soluzione univoca solamente se la matrice

$$K = \begin{bmatrix} Q_{FF} & E_F^T \\ E_F & 0 \end{bmatrix}$$

è invertibile, ovvero se ha rango pieno.

Tuttavia, se K non è invertibile, il sistema non può essere risolto. In tale situazione non si può procedere con la risoluzione standard, ma è necessario cercare una **direzione di discesa illimitata** d che soddisfi le seguenti condizioni:

- $Qd = 0$: la direzione non contribuisce alla parte quadratica della funzione obiettivo;
- $E_W d = 0$: la direzione è ammissibile, cioè rispetta i vincoli attivi;
- $q^T d < 0$: la funzione obiettivo decresce lungo tale direzione.

Se esiste una direzione d che soddisfa tutte queste condizioni, significa che il sottoproblema non possiede un minimo.

La matrice $Q_{FF} \in \mathbb{R}^{r \times r}$ rappresenta la restrizione della matrice Q alle sole variabili libere, ed è simmetrica e semidefinita positiva.

5.3. EVD: Eigenvalue Decomposition

5.3.1. 1. Il Sistema KKT Ristretto

Poiché $\mathbf{K}$ è simmetrica,possiamo usare la decomposizione degli autovalori (EVD) per analizzarne le proprietà:

```
\Lambda, V = eigh(K)
```

Questo calcola $\mathbf{\Lambda}$ (un vettore di autovalori λ_i) e $\mathbf{V}$ (una matrice i cui vettori colonna $\mathbf{v}_i$ sono i corrispondenti autovettori), tali che:

$$\mathbf{K} = \mathbf{V}\text{diag}(\mathbf{\Lambda})\mathbf{V} \qquad \text{e} \qquad \mathbf{K}\mathbf{v}_i = \lambda_i \mathbf{v}_i$$

A questo punto calcoliamo l'autovalore minimo in modulo:

$$\lambda_{\min} = \min(|\lambda_i|) \quad \forall i$$

Questo valore viene usato per valutare la singolarità:

Caso 1: $\lambda_{\min} >$ **TOL (Caso Regolare)**

Se tutti gli autovalori sono significativamente diversi da zero, $\mathbf{K}$ è invertibile (regolare).

Esiste un'unica soluzione $\mathbf{z}$ che rappresenta il passo di Newton completo verso il minimo del sottoproblema corrente.Per questo si procede a risolvere il sistema in modo standard

Caso 2: $\lambda_{\min} \leq$ TOL (Caso Singolare)

Se esiste almeno un autovalore nullo (o quasi nullo), $\mathbf{K}$ è singolare. Quindi non esiste un'unica soluzione. Potrebbero esserci infinite soluzioni (se $\mathbf{r}$ è nello spazio delle colonne di $\mathbf{K}$) o nessuna soluzione. Dobbiamo trovare una direzione di discesa $\mathbf{d}_{\mathcal{F}}$ che sia ammissibile per i vincoli (cioè $\mathbf{E}_F\mathbf{d}_{\mathcal{F}} = \mathbf{0}$) e che riduca la funzione obiettivo.

- **Fase 2a: Ricerca Spazio Nullo:**
  1. Gli autovettori $\mathbf{v}_i$, corrispondenti agli autovalori nulli ($\lambda_i \leq$ TOL), formano una base per lo spazio nullo di $\mathbf{K}$.
  2. Lo spazio nullo $\mathbf{V}_0$ si applica all'intero vettore $\mathbf{z} = [\mathbf{x}_{\mathcal{F}}; \mu]$. Si estrae solo la porzione relativa alle variabili $\mathbf{x}$, $\mathbf{D}_{\text{null}} = \mathbf{V}_0[1 : |\mathcal{F}|, :]$. Questa matrice $\mathbf{D}_{\text{null}}$ forma una base per le direzioni $\mathbf{d}_{\mathcal{F}}$ ammissibili (cioè $\mathbf{E}_F\mathbf{d}_{\mathcal{F}} = \mathbf{0}$).
  3. Si calcola il gradiente corrente $\mathbf{g}_{\mathcal{F}}$ e si cerca la direzione nello spazio nullo che scende più ripidamente. Questo si ottiene proiettando il gradiente negativo $-\mathbf{g}_{\mathcal{F}}$ sulla base $\mathbf{D}_{\text{null}}$:

$$\mathbf{d}_{\mathcal{F}}^{\text{desc}} = -\mathbf{D}_{\text{null}}(\mathbf{D}_{\text{null}}\mathbf{g}_{\mathcal{F}})$$

  4. Se $\|\mathbf{d}_{\mathcal{F}}^{\text{desc}}\| >$ TOL: È stata trovata una valida direzione di discesa. L'algoritmo usa questa direzione per fare un passo, calcola α per trovare il primo vincolo bloccante e continua all'iterazione successiva.
- **Fase 2b: Soluzione di Norma Minima:**
  - Se $\|\mathbf{d}_{\text{desc}}^{\mathbf{F}}\| \leq$ TOL: Questo significa che il gradiente è già ortogonale allo spazio nullo. Non possiamo migliorare l'obiettivo muovendoci in quelle direzioni. Siamo bloccati nel sottospazio singolare.
  - Per decidere se rilasciare un vincolo, abbiamo comunque bisogno dei moltiplicatori μ.
  - Il codice calcola la soluzione di norma minima (la più "piccola" $\mathbf{z}$ che risolve il sistema $\mathbf{Kz} = \mathbf{rhs}$) utilizzando la **Decomposizione in Autovalori (EVD) modificata**:
    1. Vengono utilizzati gli autovalori $\mathbf{\Lambda}$ e gli autovettori $\mathbf{V}$ di $\mathbf{K}$.
    2. Viene calcolato il vettore degli inversi modificati, $\mathbf{\Lambda}_{\text{inv_mod}}$, dove l'inverso è preso solo per gli autovalori $|\lambda_i| >$ threshold (con threshold $= 10^{-10}$), mentre i valori singolari ($\leq$ threshold) vengono posti a zero.
    3. La soluzione $\mathbf{z}$ viene calcolata come:

$$\mathbf{z} = \mathbf{V}\mathbf{\Lambda}_{\text{inv_mod}}\left(\mathbf{V}^{\intercal}\mathbf{rhs}\right)$$

    4. Questa operazione simula l'azione della pseudoinversa $\mathbf{K}^+$, garantendo che $\mathbf{z}$ sia la soluzione che ha la **norma minima** ($\min \|\mathbf{z}\|_2$) tra tutte le possibili soluzioni del sistema KKT singolare.
  - Questo fornisce un $\mathbf{z}$ valido (e quindi i μ) che permette all'algoritmo di procedere alla verifica delle condizioni KKT .

6. Calcolo dello Step Size

Dopo aver determinato la direzione di ricerca proiettata d^*, il passo successivo è calcolare lo *step size* per muoversi lungo questa direzione e trovare una nuova soluzione ammissibile all'interno della regione definita dai vincoli.

Il calcolo dello *step size* si semplifica grazie alla presenza dei vincoli di disuguaglianza (*box-constraints*), che definiscono esplicitamente un intervallo ammissibile per ogni componente del vettore x.

Il valore di α_k viene scelto come il più grande nell'intervallo $[0, 1]$ per cui tutti i vincoli risultano soddisfatti:

$$\alpha_k = \min\left(1, \min_{i \in \mathcal{F}, d_{k,i} \neq 0} \left\{ \begin{array}{ll} \frac{u_i - x_i}{d_{k,i}} & \text{se } d_{k,i} > 0 \\ \frac{0 - x_i}{d_{k,i}} & \text{se } d_{k,i} < 0 \end{array} \right\}\right)$$

6.1. Aggiornamento della soluzione

Una volta determinato lo *step size* ottimale, la nuova soluzione $x^{(k+1)}$ viene calcolata come:

$$x^{(k+1)} = x^{(k)} + \alpha_k d_k,$$

dove α_k è il valore dello step size calcolato precedentemente.

Questo aggiornamento assicura che la soluzione si muova nella direzione proiettata mantenendo la fattibilità rispetto ai vincoli di uguaglianza e disuguaglianza.

7. Numero finito di iterazioni

7.1. Numero finito di possibili basi (B)

In un problema con m vincoli, esiste un numero finito di possibili combinazioni di vincoli attivi, poiché ciascun vincolo può essere attivo o non attivo. Pertanto, il metodo ha un numero limitato di configurazioni possibili di B. Una volta trovato il set attivo corretto B^*, che identifica la soluzione ottima, il metodo si arresta.

7.2. Stretta decrescita della funzione obiettivo

Ad ogni iterazione, il metodo calcola una nuova soluzione $x^{(k+1)}$ che soddisfa i vincoli definiti dal set attivo corrente. Se la soluzione non è ottima, il metodo modifica B, aggiungendo o rimuovendo vincoli, per generare una nuova configurazione del problema. Grazie alla stretta convessità della funzione obiettivo,il valore di $f(x)$ decresce strettamente ad ogni iterazione (oppure rimane invariato in casi limite).

Questo implica che non è possibile tornare due volte allo stesso set attivo B, poiché altrimenti si otterrebbe lo stesso valore di $f(x)$, contraddicendo la stretta decrescita.

7.3. Condizioni di terminazione

Il metodo termina quando vengono soddisfatte le condizioni di ottimalità, ovvero:

- La soluzione trovata soddisfa tutte le condizioni di Karush–Kuhn–Tucker (KKT) per il problema;
- Non esistono più direzioni di discesa che migliorano la funzione obiettivo rispettando i vincoli.

Una volta trovato il set attivo B^* che rappresenta la configurazione corretta dei vincoli attivi alla soluzione ottima, il metodo si arresta poiché il problema è risolto.

7.4. Ruolo della stretta convessità

Se la matrice Q è definita positiva (cioè la funzione obiettivo è strettamente convessa), la soluzione ottima è unica e il valore della funzione obiettivo $f(x)$ è limitato inferiormente. La stretta convessità impedisce l'insorgere di cicli o oscillazioni infinite, garantendo un progresso verso la soluzione ottima ad ogni iterazione. Sebbene il problema considerato sia caratterizzato da una matrice Q semidefinita positiva, le stesse considerazioni rimangono valide. Inoltre, garantendo che ad ogni iterazione venga adottato uno *step size* e una direzione di discesa non nulli, si assicura che l'algoritmo non possa generare cicli e converga in un numero finito di passi.

8. Pseudocodice

Active-Set EVD Solver per QP

1. **Inizializzazione:**
   - $x \leftarrow x_{\text{start}}$ (Punto Ammissibile)
   - $\mu \leftarrow \mathbf{0}$
   - **Insiemi:** $\mathcal{L} \leftarrow \emptyset$ (Lower), $\mathcal{U} \leftarrow \emptyset$ (Upper), $\mathcal{F} \leftarrow \{\text{indici liberi}\}$
2. **Ciclo Principale (for $k = 1 \ldots$ max_iter):**
3. **Passo 2.1: Risoluzione del Sottoproblema KKT**
   (a) Costruisci il sistema KKT:

$$K \cdot \begin{pmatrix} x_{\mathcal{F}} \\ \mu \end{pmatrix} = \begin{pmatrix} -(Q_{\mathcal{F}\mathcal{F}} x_{\mathcal{F}} + q_{\mathcal{F}} + Q_{\mathcal{F}\mathcal{U}} x_{\mathcal{U}}) \\ b - E_{\mathcal{U}} x_{\mathcal{U}} \end{pmatrix}$$

   (b) **Risolvi** $Kz = \text{RHS}$ usando EVD ($K = V \Lambda V^T$):
   - **Se** $\lambda_{\min}(K) >$ **TOL (Regolare):** $d_k \leftarrow z_{\mathcal{F}}$.
   - **Se** $\lambda_{\min}(K) \leq$ **TOL (Singolare):**
     i. Cerca direzione di discesa d_{nullo} nello spazio nullo. Se $\|d_{\text{nullo}}\| > \text{TOL}$, $d_k \leftarrow d_{\text{nullo}}$. (**Vai a Passo 2.3**).
     ii. Altrimenti: Calcola $d_{\mathcal{F}}$ usando la pseudo-inversa (soluzione a norma minima).
4. **Passo 2.2: Test di Ottimalità (Se $\|d_k\| \leq$ TOL)**
   (a) Calcola Gradiente Lagrangiano: $g = Qx + q + E^T \mu$.
   (b) **Verifica KKT (Moltiplicatori):**

- Se $\exists i \in \mathcal{L}$ con $g_i < -\text{TOL}$: **Rilascia** $i \to \mathcal{F}$.
- Se $\exists i \in \mathcal{U}$ con $g_i > +\text{TOL}$: **Rilascia** $i \to \mathcal{F}$.

(c) Se **Nessun Vincolo Rilasciato**: **STOP**. Ottimo trovato.

(d) Altrimenti: **Continua** con i nuovi insiemi.

5. **Passo 2.3: Line Search e Aggiornamento (Se $\|d_k\| > \text{TOL}$)**

(a) Calcola passo massimo $\alpha_{\max}$ prima di colpire 0 o u_i.

(b) Passo effettivo: $\alpha \leftarrow \min(1.0, \alpha_{\max})$.

(c) Aggiorna: $x \leftarrow x + \alpha d_k$.

(d) **Se** $\alpha < 1.0 - \text{TOL}$**:**

- Identifica indice bloccante i_{block}.
- **Aggiungi** i_{block} a $\mathcal{L}$ o $\mathcal{U}$, **Rimuovi** da $\mathcal{F}$.

9. Risultati

9.1. Dataset Grandi

L'analisi si estende a problemi di grandi dimensioni, suddivisi in tre gruppi (Folder **1000**, **2000**, **3000**)].

9.1.1. Precisione Numerica

I risultati mostrano una notevole coerenza tra le soluzioni fornite da CVXPY e HiGHS .

Table 1: Errore Assoluto Medio (AAE) per Classe di Istanze

Classe	AAE vs CVXPY	AAE vs HiGHS
`netgen-1000`	7.8763×10^{-2}	7.8746×10^{-2}
`netgen-2000`	8.5483×10^{-5}	8.5481×10^{-5}
`netgen-3000`	9.2841×10^{-2}	9.2846×10^{-2}

9.1.2. Prestazioni Computazionali e Scalabilità

Il confronto sui tempi medi di esecuzione (in secondi) evidenzia il principale limite dell'algoritmo Custom.

Table 2: Tempo Medio di Esecuzione (Secondi)

Classe	Custom Time (s)	CVXPY Time (s)
`netgen-1000`	437.95	0.06
`netgen-2000`	12570.58	0.33
`netgen-3000`	53899.70	0.48

- **Scarsa Scalabilità:** Il tempo di esecuzione dell'algoritmo Custom **cresce drasticamente** con la dimensione del problema. Per le istanze `netgen-3000`, l'esecuzione media richiede circa **15 ore**, mentre CVXPY completa l'attività in meno di un secondo.

L'algoritmo **Custom** raggiunge un'elevata **accuratezza** e **robustezza** (tasso di successo del 100%) comparabile a quella dei benchmark.

Tuttavia, il suo impiego è limitato dalla sua **scarsa scalabilità** e dalla conseguente **inefficienza computazionale**. L'incremento esponenziale del tempo di esecuzione rende l'algoritmo Custom un **valido prototipo di accuratezza**, ma non una soluzione pratica per la risoluzione di problemi di ottimizzazione di dimensioni significative.

9.1.3. CPU Time

L'analisi dei tempi di calcolo (CPU Time) per i problemi più grandi rivela una **differenza di ordini di grandezza** nelle prestazioni, con il solver **Custom nettamente in svantaggio** e il suo tempo di esecuzione che scala in modo esponenziale con la dimensione del problema.

- **CVXPY-Gurobi** è il solver più veloce in assoluto, con tempi che rimangono nell'ordine del secondo anche per i problemi più grandi.

Table 3: Confronto del Tempo Medio di CPU (secondi)

Categoria (Folder)	Avg CPU Time CVXPY-Gurobi (s)	Avg CPU Time HiGHS (s)	Avg CPU Time Custom (s)
1000	0.081	0.622	477.77
2000	0.348	4.563	11 335.66
3000	0.953	20.203	62 135.54

9.1.4. Efficienza Algoritmica

Il confronto tra il numero di iterazioni conferma che l'inefficienza del solver Custom è legata al **costo computazionale per iterazione**.

Table 4: Confronto Iterazioni e Tempo per Iterazione

Categoria (Folder)	Avg Iterations Custom	Avg Iterations CVXPY-Gurobi	Avg Time per Iter Custom (s)
1000	1513.62	nan	0.316
2000	2484.29	1166.00	4.563
3000	3181.00	1867.33	19.533

- Il solver **Custom** generalmente richiede un **numero di iterazioni maggiore** rispetto a CVXPY-Gurobi per i Folder 2000 e 3000.
- Il fattore dominante è il **tempo medio per iterazione** del solver Custom, che aumenta drammaticamente con la dimensione del problema (da 0.316s a 19.533s).
- Questo conferma che il **collo di bottiglia principale** del solver Custom è l'elevatissimo e crescente costo di calcolo di ogni singola iterazione.

9.2. Dataset Casuali (PD e PSD)

L'analisi si estende a un set di istanze casuali con matrici Q definite positive (PD) e semidefinite positive (PSD). Il dataset è composto da 18 classi di problemi, generate variando il numero di archi $(5, 40, 100)$ e la percentuale di nodi vincolati $(10\%, 50\%, 90\%)$. Ogni classe include 50 istanze di test.

9.2.1. Precisione Numerica

Relativamente alla **Precisione Numerica** (misurata come Errore Assoluto Medio sull'Obiettivo), per i problemi con matrici **PD** l'errore è generalmente molto contenuto e allineato tra i solutori, mantenendosi nell'ordine di 10^{-12} o 10^{-11} .

Tuttavia, si riscontrano eccezioni significative nelle classi **PSD** con vincoli molto attivi (bassa percentuale di nodi), in particolare:

`PSD_40arcs_10pct_nodes`: L'errore medio tra `Custom` e i benchmark raggiunge ≈ 1.50. `PSD_100arcs_10pct_nodes` L'errore tra `Custom` e i benchmark è elevatissimo, nell'ordine di ≈ 67.77.

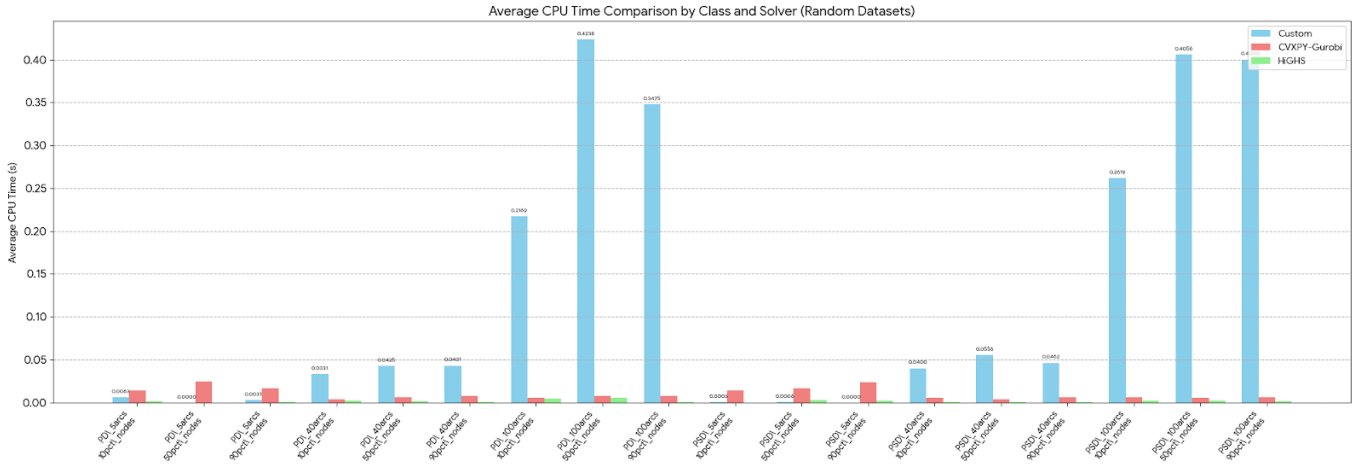

Figure 1: Precisione Numerica

9.2.2. Prestazioni Computazionali e Scalabilità

Il confronto sui **Tempi Medi di CPU (s)** rivela una chiara superiorità dei solutori di riferimento.

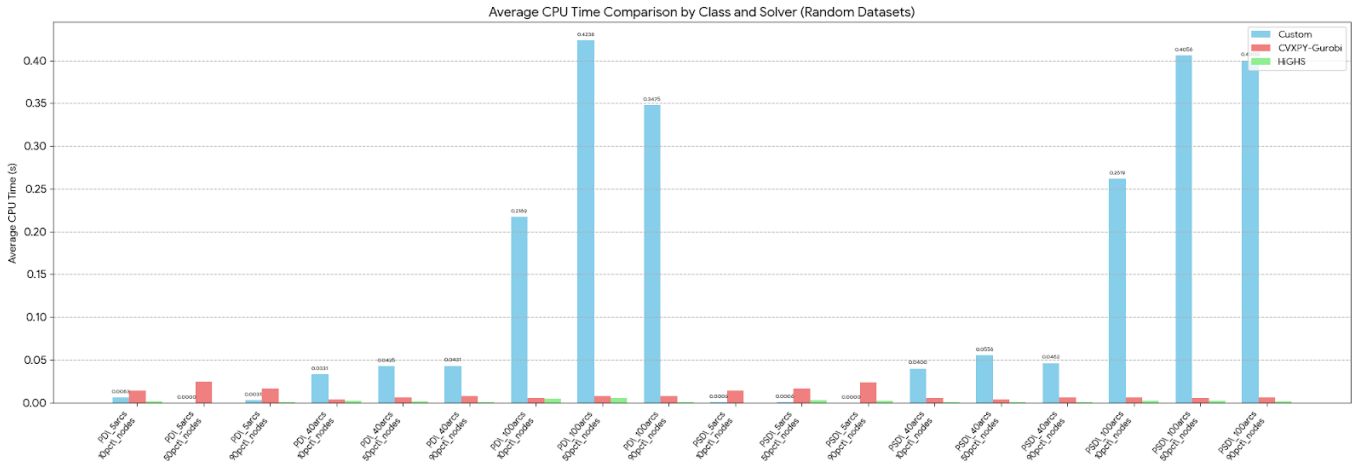

Figure 2: Tempi Medi di CPU

HiGHS si conferma il solutore **più veloce** in quasi tutte le istanze, con tempi che rimangono generalmente nell'ordine di 10^{-4} o 10^{-3} secondi. **CVXPY-Gurobi** è il secondo più veloce, posizionandosi nell'ordine di 10^{-2} secondi. Il solutore **Custom** risulta il **più lento**. Sebbene accettabile per i problemi più piccoli (5 archi), la sua inefficienza aumenta notevolmente con la dimensione: nelle istanze a 100 archi, il tempo medio di `Custom` raggiunge circa 0.4 **secondi**, un ordine di grandezza superiore ai benchmark.

Questa tendenza è più marcata nei problemi con matrici **PSD** e 100 archi , confermando una scarsa scalabilità del metodo Active Set implementato.

9.2.3. Efficienza Algoritmica

L'analisi del **Numero Medio di Iterazioni** mostra un comportamento non uniforme:

Per i problemi più grandi (100 archi PD e PSD), il solutore **Custom** esegue un **numero significativamente inferiore di iterazioni** rispetto a `CVXPY-Gurobi`. Ad esempio, per `PSD_100arcs_90pct_nodes`, `Custom` richiede 29.72 iterazioni contro le 120.00 di `CVXPY-Gurobi`.

Figure 3: Efficienza Algoritmica (Iterazioni)

Nonostante il potenziale vantaggio derivante dal minor numero di iterazioni in molti casi, il **Tempo Medio per Iterazione (s)** del solutore `Custom` si rivela il principale collo di bottiglia:

> Per i problemi con 100 archi, il tempo per iterazione di `Custom` è di circa **0.012-0.013 secondi**, mentre per i problemi a 5 archi è nell'ordine di 10^{-4} o 10^{-3} secondi.

L'elevato e crescente costo computazionale di ogni singola iterazione di `Custom`, dovuto alla risoluzione del sottoproblema quadratico mediante decomposizione EVD, annulla il beneficio di un minor numero di iterazioni rispetto a `CVXPY-Gurobi`, risultando in un tempo totale di esecuzione molto più lungo.

A. Appendice: Metodo alternativo per trovare la direzione di discesa d

A.1. Algoritmo di Pivoting Diagonale per Matrici Semidefinite Positive

Nel caso in cui la matrice Hessiana Q_{FF} (o la matrice KKT K) sia solo **Semidefinita Positiva (PSD)**, la decomposizione di Cholesky standard non è applicabile. Si utilizza la **fattorizzazione di Cholesky con pivoting diagonale** per ottenere una scomposizione a rango ridotto.

Obiettivo: Determinare una fattorizzazione del tipo $K = PVV^\top P^\top$, dove V espone il rango r della matrice K.

Lemma 4.1. (Pivoting): Per una matrice simmetrica e PSD, il valore massimo può essere raggiunto sugli elementi diagonali, garantendo che il pivoting diagonale preservi la simmetria.

Algoritmo:

- Ad ogni passo i, si cerca il massimo elemento diagonale non processato k_{r_i,r_i} (pivoting).
- Se $k_{r_i,r_i} = 0$, l'algoritmo si interrompe, indicando che la matrice non è a rango pieno.

La decomposizione risultante è a **rango ridotto**:

$$P^\top K P = VV^\top, \qquad V = \begin{bmatrix} Z & 0 \\ N & 0 \end{bmatrix}, \qquad Z \in \mathbb{R}^{r\times r} \text{ triangolare inferiore invertibile}, \quad r = \operatorname{rank}(K). \tag{7}$$

A.2. Determinazione della Direzione Infinita nel Caso Singolare

Quando la matrice KKT K è singolare, si considera il sistema omogeneo associato per trovare le direzioni illimitate (nulle) d:

$$K \begin{bmatrix} d \\ \mu \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

Qualsiasi soluzione non banale d
μ appartiene al nucleo di K e la direzione d soddisfa la condizione $d^\top Q d = 0$.

A.2.1. Determinazione della Matrice D

La fattorizzazione a rango ridotto (7) consente di esplicitare il nucleo $\mathbf{N}(\mathbf{K})$. I vettori nel nucleo sono della forma $z = Py$, con $V^\top y = 0$.

Scrivendo $y = [v; n]^\top$, la condizione $V^\top y = 0$ implica:

$$Z^\top v + N^\top n = 0 \quad \Rightarrow \quad v = -Z^{-\top} N^\top n.$$

La componente d del vettore $z = Py$ appartiene al sottospazio delle direzioni ammissibili e nulle S:

$$S = N(E) \cap N(Q),$$

dove d soddisfa $Ed = 0$ e $Qd = 0$.

Raccogliendo tali direzioni in colonne, si ottiene la matrice $D = [d_1, \ldots, d_k]$, base di S.

Direzione di Discesa Illimitata: Se il gradiente q non è ortogonale a S (i.e., $D^\top q \neq 0$), esiste una direzione $d = Dc$ che garantisce $q^\top d \neq 0$. Una scelta naturale è $d = D(D^\top q)$, che massimizza la variazione $q^\top d$ e assicura:

$$q^\top d = \|D^\top q\|^2 > 0.$$

A.2.2. Risoluzione del Sistema nel Caso Singolare

Se il sistema $Kz = r$ è consistente ma K è singolare, la soluzione z viene trovata utilizzando la fattorizzazione $P^\top KP = VV^\top$.

Separando $V = [V_1 \; V_0]$, dove V_1 è a rango pieno, la **soluzione di norma minima** si ottiene risolvendo il sistema non singolare $V_1 V_1^\top y = P^\top r$ e ponendo a zero le componenti di y corrispondenti a V_0:

$$z = Py = P \begin{bmatrix} y_1 \\ 0 \end{bmatrix}.$$

Questa procedura evita l'uso della pseudoinversa e fornisce una soluzione ortogonale al nucleo di K.

References

1. J. Wang and C. Liu, "Generating Multivariate Mixture of Normal Distributions using a Modified Cholesky Decomposition," in *Proceedings of the 2006 Winter Simulation Conference*, Monterey, CA, USA, 2006.